

Containers

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercícios

Laboratório Prático: Trabalhar com Docker Compose

Objetivo: Este laboratório irá guiá-lo através dos fundamentos da criação, gestão e implementação de aplicações usando o Docker Compose. Irá aplicar os conceitos de imagens, contentores, volumes e redes para construir e executar aplicações de serviço único e multi-serviço.

Pré-requisitos:

- Um computador com um navegador web moderno e um editor de texto.
 - Docker e Docker Compose instalados (o plugin Compose já vem incluído nas instalações recentes do Docker).
-

Instalar o Docker no Debian

Se estiver a usar um anfitrião Linux, siga estes passos no seu terminal para instalar a versão mais recente do Docker. Baseado nestas [instruções](#).

1. Configurar o repositório apt do Docker:

```
# Remover pacotes não oficiais do docker
sudo apt remove docker.io docker-doc \
docker-compose podman-docker containerd runc

# Atualizar o índice de pacotes e instalar pré-requisitos
sudo apt update
sudo apt install ca-certificates curl

# Adicionar a chave GPG oficial do Docker
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg \
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Adicionar o repositório às fontes do Apt:
echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/debian \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
```

2. Instalar os pacotes Docker:

```
sudo apt install docker-ce docker-ce-cli containerd.io \
docker-buildx-plugin docker-compose-plugin
```

3. **Gerir o Docker como um utilizador não-root (Recomendado):** Para executar comandos docker sem sudo, adicione o seu utilizador ao grupo docker.

```
sudo usermod -aG docker $USER
```

Importante: Tem de fazer logout e login novamente para que esta alteração tenha efeito. Pode verificar executando groups no terminal – o grupo docker deverá aparecer na lista.

4. **Verificar a instalação:**

```
docker --version
docker compose version
```

Ambos os comandos devem devolver informação de versão sem erros.

Dica: Se obtiver erros de permissão ao executar comandos docker, certifique-se de que fez logout e login após adicionar o seu utilizador ao grupo docker. Em alternativa, pode reiniciar a sessão com newgrp docker.

Exercício 1: “Hello, World” com Docker Compose

Objetivo: Compreender a estrutura básica de um ficheiro compose.yml e executar uma imagem pré-construída.

1. Crie uma nova pasta para este exercício e entre nela:

```
mkdir ex1-helloworld && cd ex1-helloworld
```

2. Dentro da pasta, crie um novo ficheiro chamado compose.yml com o seguinte conteúdo:

```
services:
  hello:
    image: hello-world
```

3. Execute a aplicação:

```
docker compose up
```

4. Observe o resultado. O contentor hello-world irá arrancar, imprimir a sua mensagem e depois terminar automaticamente.

5. Limpe o contentor criado pela execução:

```
docker compose down
```

Verificação: Deve ver a mensagem Hello from Docker! na saída do terminal. Após docker compose down, execute docker compose ps -a para confirmar que não restam contentores.

Dica: O comando docker compose up descarrega automaticamente a imagem se ela não existir localmente. Pode verificar as imagens locais com docker images.

Exercício 2: Construir uma Imagem de Servidor Web Personalizada

Objetivo: Usar um Dockerfile com o Docker Compose para criar uma imagem de aplicação autónoma.

1. Crie a estrutura de pastas necessária:

```
mkdir -p ex2-build/my-website && cd ex2-build
```

2. Dentro de my-website, crie um ficheiro chamado index.html:

```
<!DOCTYPE html>
<html>
<body>
  <h1>Esta pagina foi construida dentro da imagem Docker!</h1>
</body>
</html>
```

3. Na raiz da pasta ex2-build, crie um Dockerfile:

```
FROM nginx:alpine
COPY ./my-website /usr/share/nginx/html
```

4. Crie o seu ficheiro compose.yml:

```
services:
  webserver:
    build: .
    ports:
      - "8080:80"
```

5. Construa e inicie o serviço. A flag -d executa-o em segundo plano (modo *detached*):

```
docker compose up --build -d
```

6. Abra o seu navegador e navegue para `http://localhost:8080`. Deverá ver a sua página web personalizada.

7. Quando terminar, pare e remova os contentores:

```
docker compose down
```

Verificação: Após o passo 5, execute `docker compose ps` para confirmar que o serviço webserver está no estado Up. Verifique no navegador que a página é apresentada corretamente.

Dica: Se a porta 8080 já estiver em uso, pode alterá-la no `compose.yml` (por exemplo, "8081:80") e aceder em `http://localhost:8081`. Verifique portas ocupadas com `ss -tlnp | grep 8080`.

Dica: O `--build` força a reconstrução da imagem. Se alterar o Dockerfile ou os ficheiros copiados, use sempre esta flag para garantir que as alterações são aplicadas.

Exercício 3: Desenvolvimento em Tempo Real com Volumes

Objetivo: Compreender como os volumes (bind mounts) permitem alterar o conteúdo do seu site sem reconstruir a imagem.

1. Crie a estrutura de pastas:

```
mkdir -p ex3-volumes/my-website && cd ex3-volumes
```

2. Dentro de my-website, crie um ficheiro `index.html`:

```
<!DOCTYPE html>
<html>
<body>
  <h1>Pagina original servida com volumes</h1>
</body>
</html>
```

3. Crie um ficheiro `compose.yml`. Desta vez, vamos usar a imagem padrão `nginx:alpine` e montar a nossa pasta local como um volume. **Não é necessário Dockerfile.**

```
services:
  webserver:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./my-website:/usr/share/nginx/html:ro
```

4. Inicie o serviço:

```
docker compose up -d
```

5. Abra o seu navegador em `http://localhost:8080` para confirmar que está a funcionar.

6. **Atualização em Tempo Real:** Enquanto o contentor está a correr, **edite o ficheiro index.html** na sua máquina anfitriã. Altere o cabeçalho para <h1>Atualizacao em tempo real com um Volume!</h1>.
7. Guarde o ficheiro e **atualize o seu navegador** (Ctrl+F5 para forçar). A alteração aparece instantaneamente!
8. Quando terminar, limpe tudo:
docker compose down

Verificação: Após alterar o index.html, confirme que a nova versão aparece no navegador sem necessidade de reiniciar o contentor ou reconstruir a imagem.

Dica: A flag :ro (read-only) no volume impede que o contentor modifique os ficheiros no seu anfitrião. É uma boa prática de segurança em cenários de desenvolvimento.

Dica: Se a página não atualizar imediatamente, o seu navegador pode estar a usar cache. Use Ctrl+F5 (hard refresh) ou abra a página numa janela de navegação privada.

Conceito Chave: Compare este exercício com o anterior. No Exercício 2, o conteúdo foi **copiado para dentro da imagem** (ideal para produção). Aqui, o conteúdo é **montado dinamicamente** a partir do anfitrião (ideal para desenvolvimento).

Exercício 4: Conteúdo Rico em Cache com Varnish e NGINX

Objetivo: Construir uma aplicação web de duas camadas com uma cache Varnish a servir uma página web rica a partir de um backend NGINX. Explorar comunicação entre serviços e o conceito de proxy reverso.

1. Criar a Estrutura de Ficheiros:

```
mkdir -p ex4-varnish-cache/{varnish,my-dynamic-website}
cd ex4-varnish-cache
```

2. Criar o Conteúdo Web:

- Encontre um GIF animado online e guarde-o dentro de my-dynamic-website como animation.gif.
- Dentro de my-dynamic-website, crie um ficheiro index.html para exibir o GIF:

```
<!DOCTYPE html>
<html lang="pt">
<head>
  <meta charset="UTF-8">
  <title>Teste de Cache Varnish</title>
  <style>
    body { font-family: sans-serif; text-align: center; padding: 2em; }
    img { max-width: 400px; border-radius: 8px; }
  </style>
</head>
<body>
  <h1>Esta pagina esta a ser servida pela cache do Varnish!</h1>
  
</body>
</html>
```

3. Criar a Configuração do Varnish:

- Dentro da pasta varnish, crie um ficheiro chamado default.vcl. Isto diz ao Varnish onde encontrar o servidor NGINX:

```
vcl 4.1;

# O backend aponta para o serviço 'nginx' definido no compose.yml.
# O Docker resolve este nome automaticamente via DNS interno.
```

```

backend default {
    .host = "nginx";
    .port = "80";
}

```

4. Criar o Ficheiro Compose:

- Na raiz da sua pasta ex4-varnish-cache, crie o `compose.yml`:

```

services:
  cache:
    image: varnish:stable
    volumes:
      - ./varnish:/etc/varnish:ro
    ports:
      - "8080:80"
    depends_on:
      - nginx
    restart: unless-stopped

  nginx:
    image: nginx:alpine
    volumes:
      - ./my-dynamic-website:/usr/share/nginx/html:ro
    # Sem 'ports:' -- este serviço só é acessível internamente
    restart: unless-stopped

```

5. Executar e Verificar:

- Inicie os serviços:
`docker compose up -d`
- Verifique que ambos os serviços estão a correr:
`docker compose ps`
- Abra o seu navegador em `http://localhost:8080`. Deverá ver a sua página web com o GIF. O ponto-chave aqui é que foi o **Varnish** que lhe serviu a página, não o NGINX diretamente.

6. Ver a cache em ação:

- Verifique os logs do NGINX para o primeiro pedido:
`docker compose logs nginx`
- Agora, atualize a página do seu navegador várias vezes (5-10 vezes).
- Verifique os logs do nginx novamente:
`docker compose logs nginx`
- Deverá ver **poucos ou nenhuns novos registos de log**, porque o Varnish está a servir o conteúdo da sua cache sem contactar o backend NGINX.

7. Limpar: `bash docker compose down`

Verificação: Compare os logs do NGINX antes e depois de atualizar a página várias vezes. Se a cache estiver a funcionar, o número de pedidos registados pelo NGINX não deverá aumentar significativamente.

Dica: O `depends_on` garante que o NGINX arranca antes do Varnish, mas **não garante que o NGINX esteja pronto a servir pedidos**. Para verificações de saúde mais robustas, pode usar a diretiva `healthcheck` no Compose.

Dica: Note que o serviço `nginx` **não expõe portas** para o anfitrião. Só é acessível a partir de outros contentores na mesma rede Docker. O Compose cria automaticamente uma rede partilhada entre todos os serviços definidos no mesmo ficheiro.

Conceito Chave: Este exercício demonstra o padrão de **proxy reverso** e **service discovery**. O Varnish encontra o NGINX pelo nome do serviço (`nginx`), graças ao DNS interno do Docker. Este é um padrão fundamental em arquiteturas web modernas.

Exercício 5: Implementar uma Aplicação do Mundo Real

Objetivo: Aprender a ler documentação oficial e a implementar um serviço complexo auto-hospedado à sua escolha usando Docker Compose.

1. **Escolha um Serviço:** Vá a [LinuxServer.io](https://lscr.io/linuxserver) e navegue pela lista de imagens populares. Escolha uma que lhe interesse, por exemplo:
 - **Jellyfin:** Um servidor de multimédia para os seus filmes e música.
 - **Nextcloud:** Uma nuvem pessoal para ficheiros, contactos e calendários.
 - **Home Assistant:** Uma plataforma de automação residencial de código aberto.
2. **Leia a Documentação:** Na página da imagem escolhida, encontre a secção “Docker Compose”. Leia-a com atenção, prestando especial atenção aos **volumes** e **variáveis de ambiente** necessários.
 - **Volumes (- ./config:/config):** É aqui que a configuração da aplicação será armazenada na sua máquina anfitriã. Isto garante que os dados persistem mesmo que o contentor seja removido.
 - **Variáveis de Ambiente (PUID, PGID, TZ):** Estas são críticas. TZ define o seu fuso horário (p. ex., Europe/Lisbon). PUID e PGID garantem que os ficheiros criados pelo contentor têm a propriedade correta. Em Linux, encontre o seu ID executando o comando `id` no seu terminal. Um valor comum é `1000`.

3. **Crie o seu compose.yml:** Com base na documentação, crie o ficheiro. Aqui está um exemplo para o Jellyfin:

```
services:
  jellyfin:
    image: lscr.io/linuxserver/jellyfin:latest
    container_name: jellyfin
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=Europe/Lisbon
    volumes:
      - ./config:/config
      - ./series:/data/tvshows
      - ./filmes:/data/movies
    ports:
      - "8096:8096"
    restart: unless-stopped
```

4. **Prepare e Implemente:**

- Crie as pastas locais que definiu nos seus volumes:

```
mkdir -p config series filmes
```

- Execute a aplicação:

```
docker compose up -d
```

- Verifique que o serviço está a correr:

```
docker compose ps
```

```
docker compose logs -f
```

(Use Ctrl+C para sair dos logs.)

5. **Explore:** Verifique a documentação para o número da porta padrão. Para o Jellyfin, é 8096. Abra o seu navegador em `http://localhost:8096` e siga o assistente de configuração para o seu novo serviço!

6. **Quando terminar:** `bash docker compose down`

Verificação: O serviço deve estar acessível no navegador na porta indicada. Execute `docker compose ps` para confirmar que o estado é Up e healthy (se disponível).

Dica: Se a aplicação demorar a arrancar, consulte os logs com `docker compose logs -f nome_do_servico` para ver o progresso. Muitas aplicações precisam de algum tempo na primeira execução para inicializar a base de dados ou configuração.

Dica: A opção `restart: unless-stopped` garante que o contentor reinicia automaticamente após um crash ou reinício do sistema, a menos que o tenha parado explicitamente com `docker compose stop`.

Referencia Rapida: Comandos Docker Compose

Utilize esta tabela como referência durante os exercícios.

Comando	Descrição
<code>docker compose up -d</code>	Inicia todos os serviços em segundo plano
<code>docker compose up --build -d</code>	Reconstrói imagens e inicia os serviços
<code>docker compose down</code>	Para e remove contentores, redes
<code>docker compose down -v</code>	Idem, e também remove volumes nomeados
<code>docker compose ps</code>	Lista os serviços e o seu estado
<code>docker compose ps -a</code>	Lista todos os serviços (incluindo parados)
<code>docker compose logs</code>	Mostra os logs de todos os serviços
<code>docker compose logs -f servico</code>	Segue os logs de um serviço específico
<code>docker compose exec servico sh</code>	Abre um shell dentro de um contentor
<code>docker compose restart servico</code>	Reinicia um serviço específico
<code>docker compose pull</code>	Descarrega as versões mais recentes das imagens
<code>docker compose config</code>	Valida e mostra a configuração final

Resolucao de Problemas Comuns

O contentor não arranca ou sai imediatamente:

- Verifique os logs: `docker compose logs nome_do_servico`
- Confirme que o `compose.yml` não tem erros de sintaxe: `docker compose config`
- Verifique se a imagem existe e é válida: `docker compose pull`

Erro “port is already allocated”:

- Outra aplicação (ou contentor) está a usar a mesma porta.
- Identifique o processo: `ss -tlnp | grep NUMERO_DA_PORTA`
- Altere o mapeamento de portas no `compose.yml` (lado esquerdo do :) para uma porta livre.

Erro “permission denied” ao executar docker:

- Certifique-se de que o seu utilizador está no grupo `docker`: `groups`
- Se acabou de se adicionar ao grupo, faça `logout` e `login` novamente.
- Em alternativa, use `sudo` antes do comando (não recomendado como solução permanente).

Alterações no `index.html` não aparecem no navegador:

- Limpe a cache do navegador com `Ctrl+F5` (hard refresh).
- Verifique que o caminho do volume no `compose.yml` está correto e aponta para a pasta certa.
- Confirme que está a editar o ficheiro correto (não uma cópia).

Erro “no such service” ou “service not found”:

- Verifique que está na pasta correta que contém o `compose.yml`.
- Confirme que o nome do serviço no comando corresponde ao definido no ficheiro.